Lab 06: Regression and Classification
Reflection Journals

ITAI 2373

Group: FA26_ML_6171_16347 5

Person 1: Kristina Akpan

Name: Kristina Akpan

Role: Person 1: Part 1, Regression vs. Classification, Knowledge Check 1, and final notebook assembly

This lab was about regression and classification, and honestly the main idea clicked faster than I expected. Regression is when you want a number back, and classification is when you want a category back. That's it. The part that helped me the most was that we used the same Titanic dataset for both. We didn't change the data, we just changed what we were trying to predict. Fare is a number, so that's regression. Survived is a 0 or a 1, so that's classification. Once I saw that, the definitions in the notebook made a lot more sense to me.

My section was Part 1 and the first Knowledge Check question. I wanted my answer to be something a person could read once and get. So instead of just repeating the textbook lines about continuous values and discrete labels, I used the question "how much" vs. "which one." If you're asking how much, it's regression. If you're asking which one, it's classification. I think that's an easy way to check yourself when you get a new problem and you're not sure which kind it is. I also wrote a small code cell to prove it with our data. Fare had 248 different values and Survived only had 2. Seeing those two numbers side by side made the difference really obvious, way more than reading the definitions.

The biggest challenge for me wasn't the content, it was the file. When I opened the notebook, every code cell was jammed onto one line. Everything was there, but the line breaks were missing, so none of it would run. I had to fix that first before anyone's code could even be tested. I made sure not to change any of the actual text from the instructor, just the line breaks. Then I put everyone's parts in, ran the whole thing from the top, and checked the outputs. That took more time than my actual section did, but it had to be done or we would have had nothing to turn in.

Running everything myself also made me look at the results closer than I would have if I just did my piece. The regression model was off by about 58 dollars on average, which is

a lot when most people paid way less than that. The classification model got about 75% right. At first 75% sounded kind of low to me, but when Thomas added the baseline, I saw that just guessing "did not survive" for everyone gets you about 61%. So the model is actually doing something useful. I wouldn't have thought to compare it like that, and I'm going to keep that in mind for future labs. A number by itself doesn't really tell you if it's good.

Something I'll take away from this is how much the metric depends on the type of problem. You can't use accuracy for fare because the model is never going to guess the exact price down to the cent. That seems obvious now, but I don't think I would have said it that clearly before this lab.

This also connects to what we've been doing in NLP. In the earlier labs we were turning text into things a computer can use, like tags and counts. This lab showed the next step, which is actually feeding features into a model and getting a prediction out. Something like spam detection or sentiment is just classification on text. The Sex column being turned into 0 and 1 is the same idea as turning words into numbers.

If I did this again, I would check the notebook file as soon as we get it, before we split up the work. Finding the formatting problem at the end made things more stressful than they needed to be. Overall, I feel a lot more confident about telling regression and classification apart, and I understand why picking the right metric matters.

Person 2: Angel Martinez

Name: Angel Martinez

Role: Person 2: Part 2, Linear Regression setup and code (Task 1)

Lab 06 was my first time building a linear regression model from start to finish in scikit learn, and the most useful thing I took from it was how consistent the workflow is. Every step followed the same pattern: create the model object, call fit on the training data, call predict on the test data, and pass the results to a metric function. Four lines of real code produced a working model. Seeing how little code was required made me realize that the hard part of machine learning is not writing the model, but preparing the data correctly and knowing how to evaluate the result.

My task was Task 1. The model was trained on two features, Age and Pclass, to predict Fare. Because the setup cell had already split the data with a test size of 0.2 and a random state of 42, my job was to make sure I used the correct variables in the correct places. It would be easy to accidentally predict on X_train_reg instead of X_test_reg, which would make the model look better than it really is, since it would be tested on data it had already seen. I double checked that the predictions came from the test set only.

I ran into two problems with the starter code. The first was a NameError. The last print statement used np.sqrt, but the notebook never imported NumPy. The fix was one line, but it reminded me that starter code is not guaranteed to run, and that it is worth reading every line rather than just uncommenting it. The second problem was more interesting. The comment said the model would be off by about 48 dollars on average. When I ran it, the square root of the MSE was 58.01. I considered whether I had made a mistake, so I reran it and checked the split, but the number was consistent. The comment was most likely written for a different version of the data or a different split. I chose to have the print statement calculate the dollar value from the actual result instead of hard coding a number that was not true.

I also printed the coefficients and the intercept so Kerry could interpret them. I included the list of feature names in the same output, because coef_ is just an array of numbers and it does not label which value belongs to which feature. Without the feature list, it would be possible to mix up the Age and Pclass slopes.

Looking at the result as a programmer, an RMSE of 58 dollars told me the model is weak. Pclass is stored as 1, 2, or 3, and the model treats it as a regular number, which assumes the price gap between first and second class is the same as between second and third. That is probably not true. If I were to improve the model, I would try one hot

encoding Pclass, adding more features such as the number of family members aboard, or using a log transform on Fare to reduce the effect of the very expensive tickets.

This lab connects to our NLP work because the same fit and predict structure applies regardless of the input. Whether the features come from text or from a spreadsheet, the model only sees numbers. Learning the pattern here on simple numeric data will make it easier to apply it later to text features like word counts or TF IDF vectors.

Person 3: Kerry Walker

Name: Kerry Walker

Role: Person 3: Linear Regression interpretation and Knowledge Check 2 and 3

I was assigned the interpretation of the linear regression results and the last two Knowledge Check questions, and I ended up spending most of my time thinking about what the numbers actually mean rather than just reporting them. I think that was the real lesson of this lab for me: a model always gives you an output, but the output is only useful if you can explain it in plain language and judge whether it is any good.

The coefficients were the most satisfying part. The model learned a slope of about negative 38.30 for Pclass and about negative 0.44 for Age, with an intercept of about 133.31. Reading those as slopes, the story is clear. Moving down one class lowers the predicted fare by around 38 dollars, while getting a year older lowers it by less than half a dollar. Passenger class does nearly all the work. I found it interesting that a single line of numbers could summarize the entire relationship the model discovered, and that the sign and size each carry a different meaning. The sign tells you the direction of the effect, and the size tells you how strong it is, as long as you keep in mind that the features are measured on different scales.

The mean squared error was harder to make sense of at first. A value of 3364.92 looks enormous, but that is because MSE is measured in squared dollars, which is not a unit anyone thinks in. Taking the square root gave about 58 dollars, which is understandable. Then I wanted context, so I looked at the test set itself. The average fare was about 33 dollars and the median was about 14 dollars. That comparison changed how I saw the model. Being off by 58 dollars when a typical ticket cost 14 dollars is not a small error. I also calculated the R squared, which came out to about 0.19, meaning the two features explain only about a fifth of the variation in fares.

Part of the reason for the large error is the way MSE works. Because each error is squared, one huge miss counts far more than several small ones. A few first class tickets in this data cost over 500 dollars, and a straight line cannot reach them. Those passengers pull the MSE up dramatically. That was an important realization for me: the choice of metric shapes what "good" looks like. MSE punishes big mistakes heavily, which can be what you want in some situations and misleading in others.

Question 3 tied all of this together. Accuracy only asks whether a prediction is exactly right, and for a continuous value like fare, a model will essentially never land on the exact amount. A prediction of 30.10 for a 30.00 ticket would count as wrong in the same way as a prediction of 300.00. Accuracy would therefore sit near zero and hide the

difference between close and far, which is the very thing we care about with numeric predictions.

The challenge for me was resisting the urge to call the model good just because it ran. Going forward, I want to always compare a metric against something meaningful, whether it is the average value, a baseline model, or a simple guess. In our NLP work this applies directly, because when we eventually predict scores or ratings from text, we will need the same habit of asking not only how big the error is, but how big it is compared to what.

Person 4: Thomas Jaymon

Name: Thomas Jaymon

Role: Person 4: Part 3, Logistic Regression setup and code (Task 2)

Okay, I'll admit it: the name "logistic regression" confused me going into this lab. Regression is supposed to predict numbers, right? So why is this a classification model? The notebook explained it, but it really clicked when I actually built it. Logistic regression calculates a probability, a number between 0 and 1, and then it turns that probability into a class. If the chance of survival is above 50%, it predicts 1. Below that, it predicts 0. So it kind of does both, which is probably why it got that name.

My job was Task 2, writing the logistic regression code. After seeing Angel's linear regression cell, mine felt pretty familiar, which was honestly a relief. Same steps: make the model, fit it, predict, and score it. The only real differences were the model type, the features (Age, Pclass, and Sex), and using accuracy_score instead of mean squared error. Our accuracy came out to 74.83%, which sounded decent, but I wasn't sure. Is 75% good? Is it bad? I didn't know how to answer that from one number alone.

So I added some extra output, and that ended up being my favorite part of the lab. First, I printed a baseline. About 60.84% of the test passengers didn't survive, so if the model just guessed "didn't survive" for everyone, it would already be right 60.84% of the time. Suddenly 74.83% looked a lot more meaningful, because the model was clearly doing better than a lazy guess. Then I printed the confusion matrix, which showed where the mistakes were. The model correctly caught 68 of the 87 people who died, but only 39 of the 56 who survived. So it's better at spotting who didn't make it than who did.

I also printed the coefficients, and wow, the Sex coefficient was way bigger than the others at about 2.53. That lines up with the "women and children first" history of the Titanic. Pclass was negative, so lower classes had worse odds, and Age was only slightly negative. It was kind of eerie to see a model rediscover a piece of history from just three columns of data. It also made me think about how these models just learn patterns from whatever they're given, including patterns that come from unfair situations.

The one tricky part was a step I didn't write but had to understand. The Sex column had to be changed from "male" and "female" into 0 and 1 before the model could use it. Models only understand numbers! That small mapping line is actually really important. If it had been skipped, the code would have crashed.

That connects straight back to our NLP labs. Text has to be turned into numbers too before a model can do anything with it. Classifying a movie review as positive or

negative is the same kind of problem we solved here, just with words instead of ages and ticket classes. Logistic regression is actually a common choice for that, so I feel like I'll be seeing this model again soon. Next time I build a classifier, I'm definitely going to print a baseline and a confusion matrix right away instead of trusting the accuracy by itself.

Person 5: Jamie Lopez

Name: Jamie Lopez

Role: Person 5: Logistic Regression interpretation and final reflection

When I sat down to interpret the classification results, I realized that every number in the output stood for a person. The test set held 143 passengers. The model predicted the fate of 107 of them correctly and 36 incorrectly. It is easy to forget that in a lab, because the data arrives as rows and columns, but the Titanic data is a record of who lived and who died, and that gave this work a weight I did not expect from a lab about linear models.

My task was to explain what the 74.83% accuracy means and to write the group's final reflection. Accuracy is a simple idea, the share of predictions that turned out right, but its simplicity can be misleading. Thanks to the baseline Thomas printed, I could show that a model that always guessed "did not survive" would already reach 60.84%. Against that floor, our model's gain of about 14 points is real but modest. The confusion matrix then told a fuller story. Of the 56 passengers who survived, the model failed to recognize 17. In other words, its blind spot was survival itself. If a model like this were used in a real situation where the positive case matters most, such as screening for a disease, missing those cases would matter far more than the headline accuracy suggests.

The coefficients were the part that stayed with me. The largest by far belonged to Sex, at about 2.53, followed by a negative weight on Pclass. The model, given only three columns, found the pattern historians describe: women and passengers in higher classes were far more likely to survive. That is a sign the model works. It is also a reminder that models learn the world as it was recorded, including its inequalities. A model trained on historical hiring or lending data would learn those patterns just as faithfully, and it would not know that some of them are unjust.

Writing the group reflection asked me to step back and look at the lab as a whole. The challenges we faced were small but telling. The starter code was missing an import, and a comment promised an error of about 48 dollars when our model actually produced about 58. Neither problem was difficult to fix, but both taught us to trust results over expectations. The insight I valued most was that a metric only becomes meaningful when it is placed next to something, whether that is a baseline, the average fare, or the cost of a particular kind of mistake.

This lab also helped me see where our NLP coursework is heading. In earlier labs we broke text apart into tokens and tags, turning language into something a computer can

count. Logistic regression is one of the classic tools that takes those counts and makes a decision, labeling a review as positive or negative or an email as spam. The same questions will follow us there: how often is the model right, where does it fail, and whose patterns has it learned?

I come away from this lab with more respect for interpretation as a skill. Training the model took a few lines. Understanding what it had learned, and what it had missed, took far more care, and I think that is where the real work of machine learning lives.